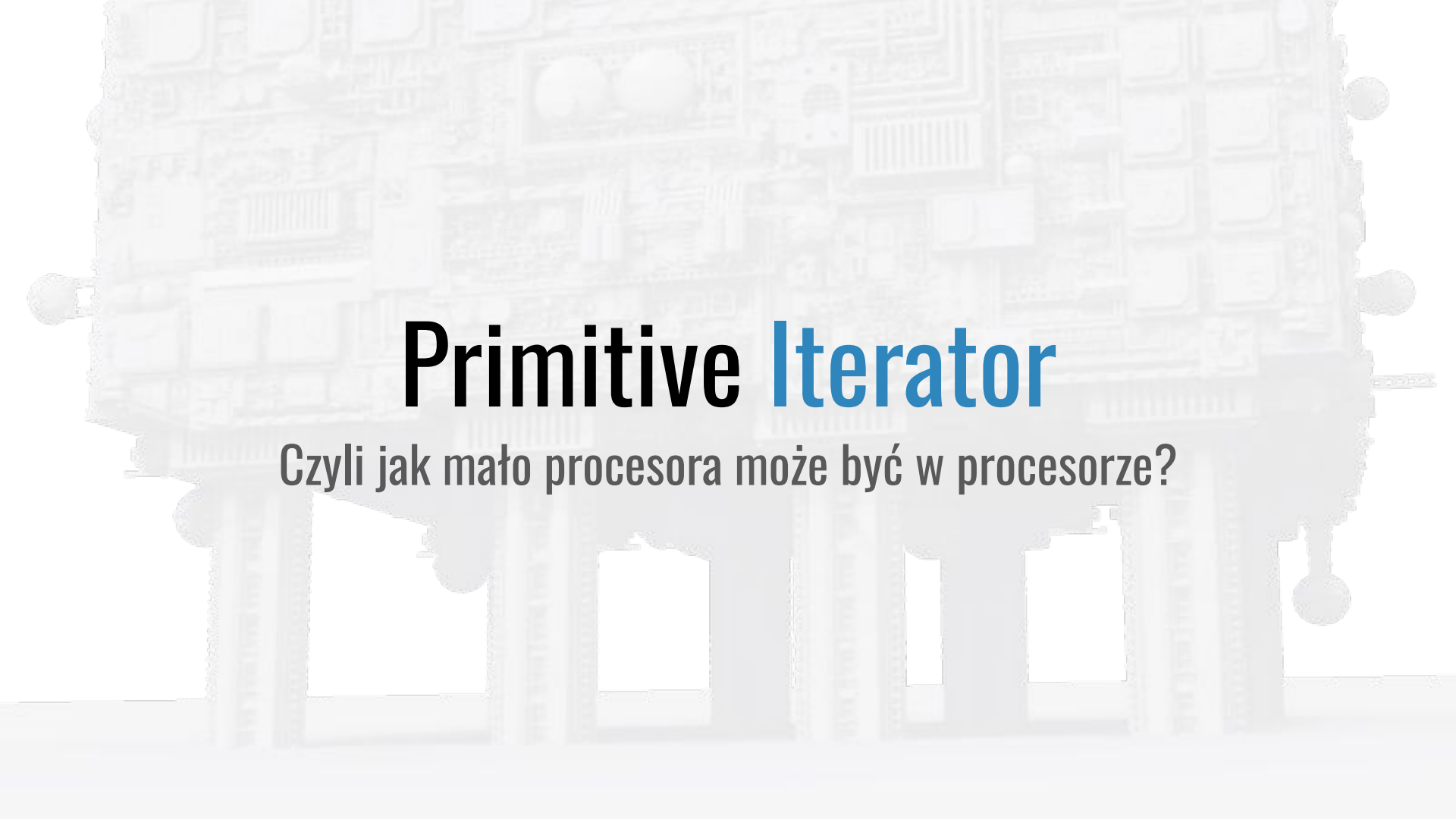




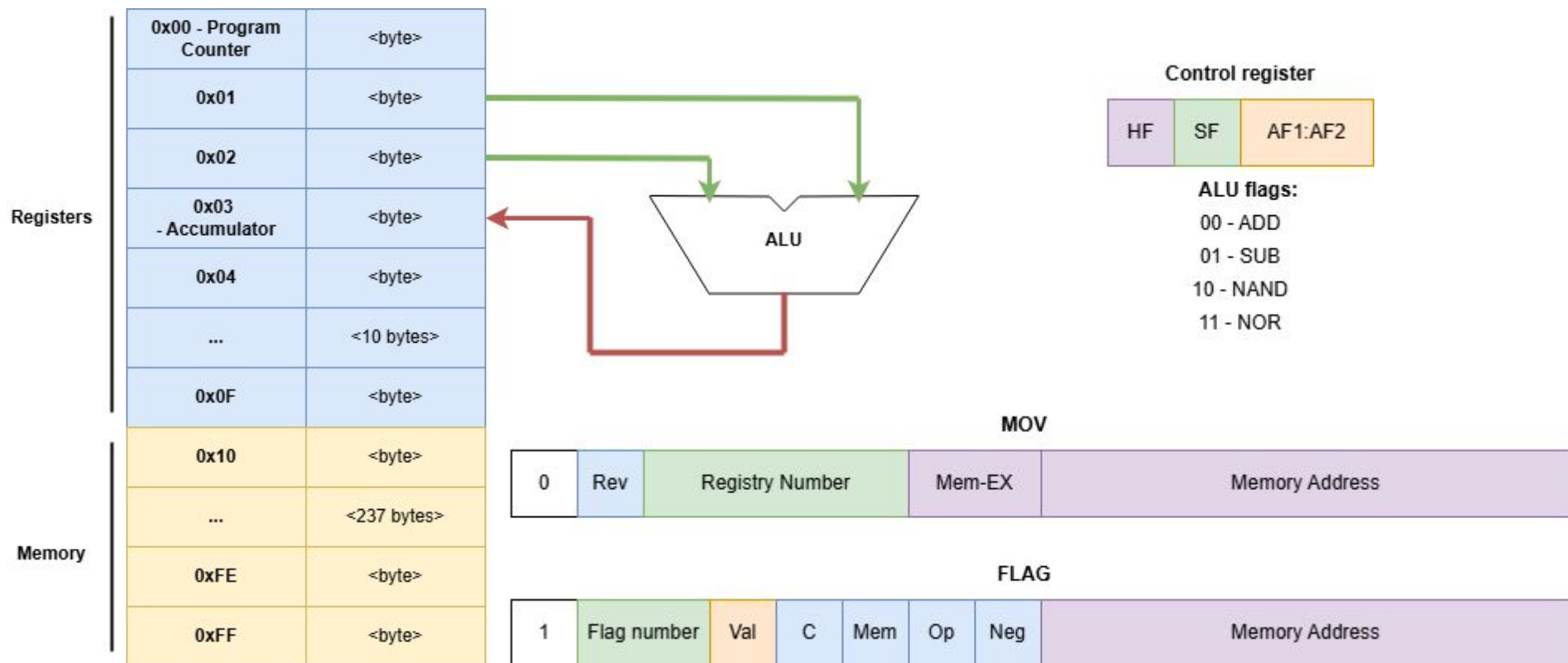
Primitive **I**terator

Czyli jak mało procesora może być w procesorze?

Podstawowe założenia projektowe

- Jak najmniej bramek logicznych w procesorze (najważniejsze)
- Jak najmniej instrukcji
- Jak najprostsza konstrukcja rdzenia

TLDR: ISA PI na jednym obrazku



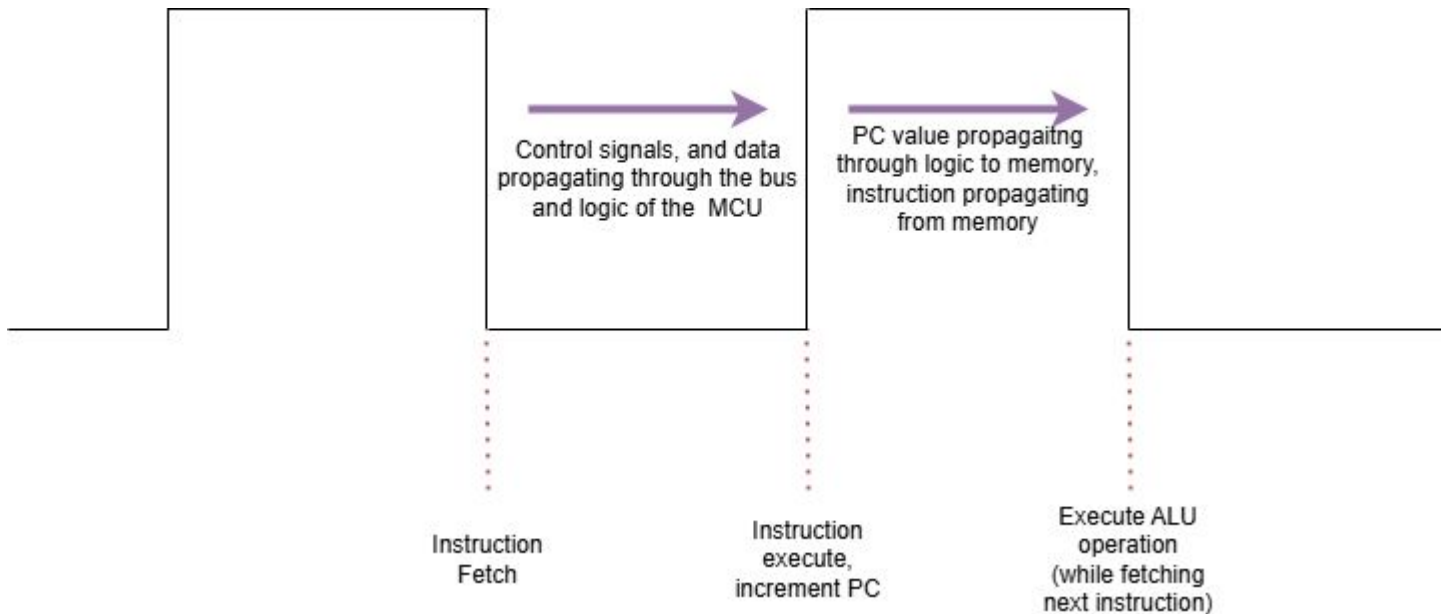
Assembler

//Function for multiplication of two 8-bit numbers. Arguments: R1, R2. Return Value: R3;

```
mul_ret: 0;
mul_iterator: 0;
mul_sum: 0;
mul_a: 0;
mul_minus1: 255;
mul_zero: 0;
mul:
    MOV 3, mul_zero;
    FLAG 1, 1;
    MOV 0, mul_ret;
    FLAG 1, 2;
    MOV 0, mul_ret;
    FLAG 1, false;
    MOV mul_sum, 3;
    MOV mul_a, 2;
    mul_ptl:
        MOV 2, mul_minus1;
        MOV mul_iterator, 3;
        MOV 1, mul_sum;
        MOV 2, mul_a;
        MOV mul_sum, 3;
        MOV 1, mul_iterator;
        FLAG 1, ~mul_iterator;
        MOV 0, mul_ptl_jump;
        FLAG 1, false;
        MOV 3, mul_sum;
    MOV 0, mul_ret;
mul_func: mul;
mul_ptl_jump: mul_ptl;
```

```
MOV 1, delay_ind;
MOV 2, delay_ptr_minus;
MOV arr2, 3;
MOV 4, >arr2;    //delay[de
```

Implementacja w VHDL



Implementacja w VHDL - dekodery instrukcji / rejestr instrukcji

```
SIGNAL state: STD_LOGIC_VECTOR(15 downto 0) := "0000000000000000";

setter: process(clk) begin
    if falling_edge(clk) then
        state <= input;
    end if;
end process setter;

opcode <= state(15);
mov_reverse <= state(14);
mov_registry_number <= state(13 downto 10);
flag_flag_number <= state(14 downto 13);
flag_value <= state(12);
flag_carry <= state(11);
flag_memory <= state(10);
flag_operation <= state(9);
flag_negate <= state(8);
address <= state(7 downto 0);

entity ir is
    Port ( input : in  STD_LOGIC_VECTOR (15 downto 0);
          opcode : out  STD_LOGIC;
          mov_reverse : out  STD_LOGIC;
          mov_registry_number : out  STD_LOGIC_VECTOR (3 downto 0);
          flag_flag_number : out  STD_LOGIC_VECTOR (1 downto 0);
          flag_negate : out  STD_LOGIC;
          flag_carry : out  STD_LOGIC;
          flag_memory : out  STD_LOGIC;
          flag_operation : out  STD_LOGIC;
          flag_value : out  STD_LOGIC;
          address : out  STD_LOGIC_VECTOR (7 downto 0);
          clk : in  STD_LOGIC);
end ir;
```

Implementacja w VHDL - rejestry ogólnego przeznaczenia

```
SIGNAL state: STD_LOGIC_VECTOR(95 downto 0) := initial;
```

```
setter: process(clk) begin
    if rising_edge(clk) then
        if set = '1' then
            state(input_address*8 + 7 downto input_address*8) <= input;
        end if;
    end if;
end process setter;

output(15 downto 8) <= state(output_address*8 + 7 downto output_address*8);
output(7 downto 0) <= state(output_address*8 + 15 downto output_address*8 + 8);
```

Implementacja w VHDL - rejestr Accumulator

```
stager: process(clk) begin
    if rising_edge(clk) then
        if set = '1' then
            staging <= input;
        end if;
    end if;
end process stager;

setter: process(clk) begin
    if falling_edge(clk) then
        if set_alu = '1' then
            state <= input_alu(7 downto 0);
            carry_state <= input_alu(8);
        else
            if set_this_cycle = '1' then
                state <= staging;
            end if;
        end if;
        if set_async = '1' then
            state <= input_async;
        end if;
    end if;
end process setter;
```


Implementacja w VHDL - Program Counter

```
setter: process(clk) begin
    if rising_edge(clk) then
        if set = '1' then
            state <= input;
        else
            if iterate = '1' then
                state <= std_logic_vector(to_unsigned(to_integer(unsigned(state)) + 2, state'length));
            end if;
        end if;
    end if;
end process setter;
```

Implementacja w VHDL - logika w bloku rejestrów

```
output_address_trimmed <= output_address when (output_address MOD 2 = 0) else output_address - 1;

full_output(7 downto 0) <= gpr_out(7 downto 0) when output_address_trimmed > 3 else
    alpha_out when output_address_trimmed = 0 else
    acc_out;

full_output(15 downto 8) <= gpr_out(15 downto 8) when output_address_trimmed > 3 else
    pc_out when output_address_trimmed = 0 else
    beta_out;

ir_output <= full_output;

output <= full_output(15 downto 8) when (output_address MOD 2 = 0) else full_output(7 downto 0);

pc_set <= set when input_address = 0 else '0';
alpha_set <= set when input_address = 1 else '0';
beta_set <= set when input_address = 2 else '0';
acc_set <= set when input_address = 3 else '0';
gpr_set <= set when input_address > 3 else '0';

gpr_in_adr <= input_address - 4 when input_address > 3 else 0;
gpr_out_adr <= output_address_trimmed - 4 when output_address_trimmed > 3 else 0;

alpha <= alpha_out;
beta <= beta_out;
pc <= pc_out;
acc <= acc_out;
```

Implementacja w VHDL - Pamięć

```
architecture mem of memory is

    SIGNAL state: STD_LOGIC_VECTOR(1919 downto 0) := initial;
    SIGNAL address_trimmed: integer;

begin
    setter: process(clk) begin
        if rising_edge(clk) then
            if set = '1' then
                state(address*8 + 7 downto address*8) <= input;
            end if;
        end if;
    end process setter;

    address_trimmed <= address when (address MOD 2 = 0) else (address - 1);

    ir_output(15 downto 8) <= state(address_trimmed*8 + 7 downto address_trimmed*8);
    ir_output(7 downto 0) <= state(address_trimmed*8 + 15 downto address_trimmed*8 + 8);

    output <= state(address*8+7 downto address*8);
end mem;
```

Implementacja w VHDL - flagi

```
setter: process(clk, resume) begin
    if rising_edge(clk) then
        if set = '1' then
            state(flag_number) <= final;
        end if;
    end if;
    if resume = '1' then
        state(0) <= '0';
    end if;
end process setter;
```

```
hf <= state(0);
sf <= state(1);
af1 <= state(2);
af2 <= state(3);
```

Implementacja w VHDL - logika rdzenia

```
rq_set_memory <= '1' when (opcode = '0' and (mov_reverse='1' and ir_address>15)) else '0';
rq_set_register <= '0' when (opcode = '1' or (mov_reverse='1' and ir_address>15)) else '1';
rq_set_flag <= opcode;

iterate <= not hf;

execute <= not sf;

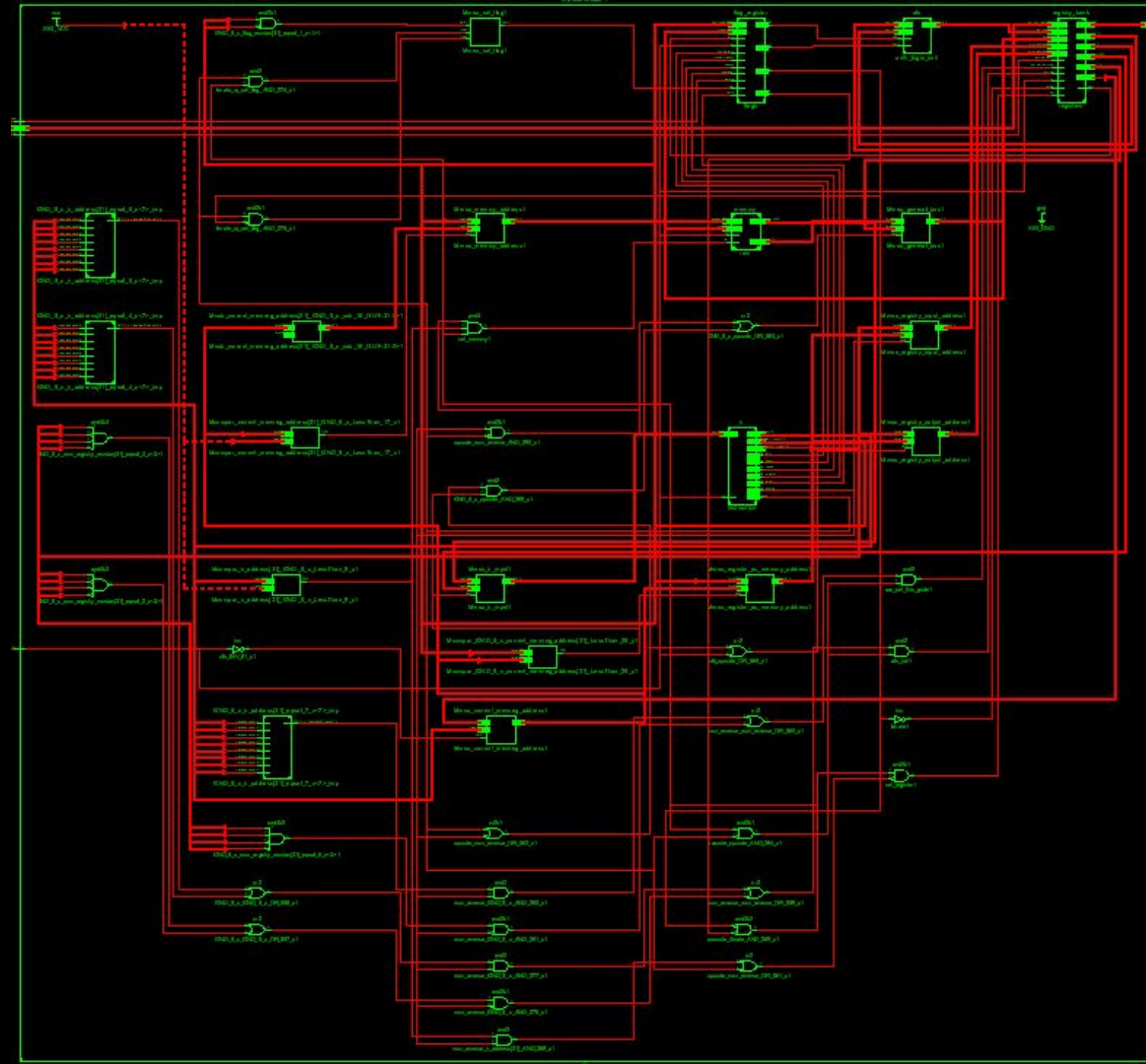
general_bus <= register_output when ((current_memreg_address < 16 and (opcode = '1' or mov_reverse = '0')) or (opcode = '0' and mov_reverse = '1')) else memory_output;

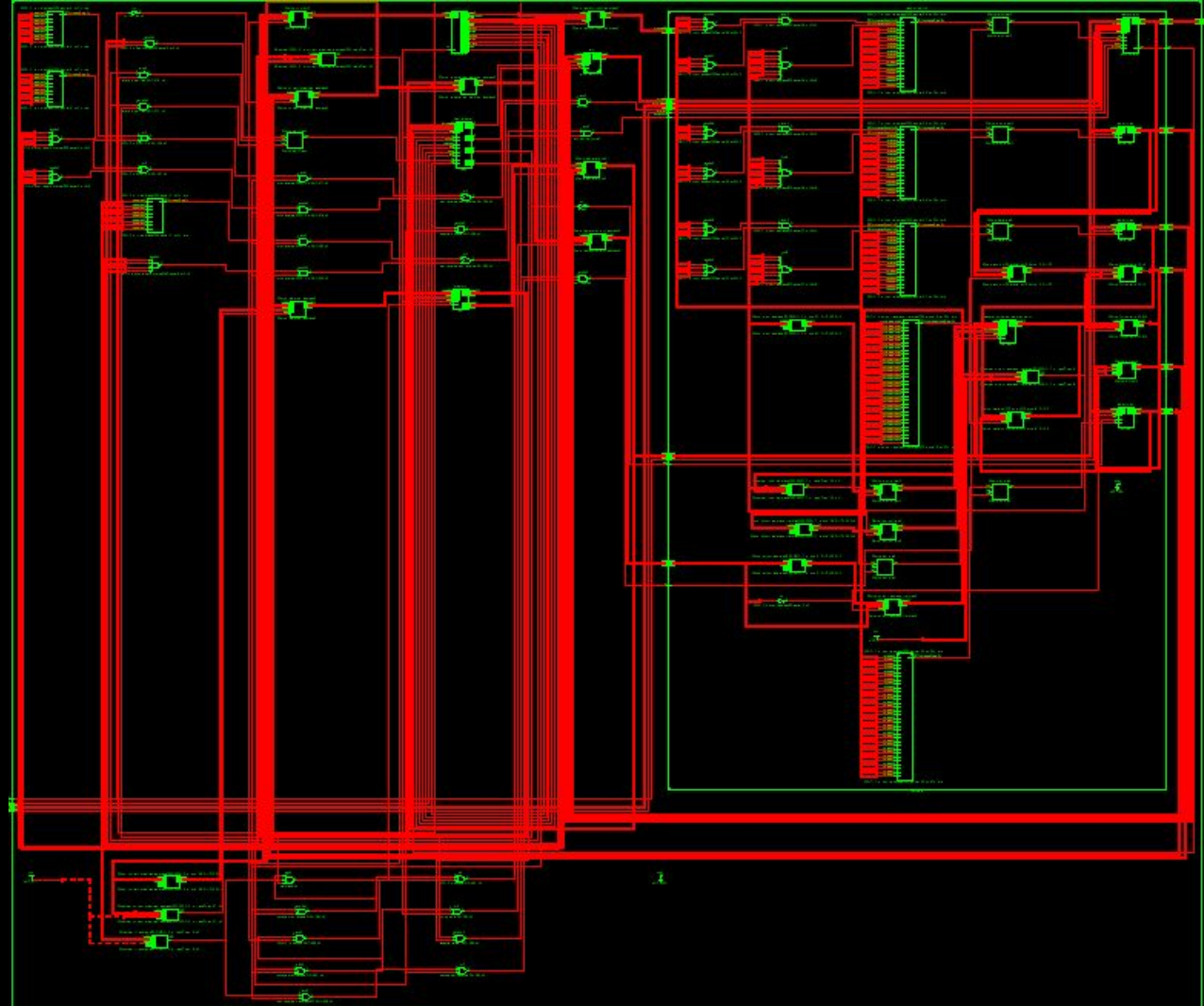
memory_address <= current_memreg_address - 16 when current_memreg_address > 15 else 0;

register_as_memory_address <= current_memreg_address when current_memreg_address < 16 else 0;

current_memreg_address <= ir_address when clk = '0' else pc;
registry_output_address <= register_as_memory_address when clk = '1' or (opcode='1' or mov_reverse='0') else mov_registry_number;
registry_input_address <= register_as_memory_address when mov_reverse = '1' else mov_registry_number;

ir_input <= ir_register_output when current_memreg_address < 16 else ir_memory_output;
```





Pls daj gwiazdkę...

<https://github.com/player400/pi>

